

Day 32 — Tue 6 Oct

Countdown: 4 days to CS Degree Paper-I

Method: start by recalling yesterday's topic with this page closed. Study the sections below, work every example by hand, then do 20 MCQs.

☉ Degree — dynamic programming & NP

Source: ComputerScience_Degree/CSD_01_AlgorithmsAndAnalysis.md

6.1 Fractional knapsack

Problem. n items, item i has value v_i and weight w_i . Knapsack capacity W . You may take **fractions** of an item. Maximise total value.

Greedy criterion: highest value-per-unit-weight (v_i/w_i) first.

Worked example. $W = 50$; items:

Item	Value v_i	Weight w_i	Ratio v_i/w_i
1	60	10	6.0
2	100	20	5.0
3	120	30	4.0

Sort by ratio: 1, 2, 3 (already sorted).

Step	Item	Capacity left before	Taken	Value added	Capacity left after
1	1	50	all 10 kg	60	40
2	2	40	all 20 kg	100	20
3	3	20	$20/30 = 2/3$ of it	$(2/3) \times 120 = 80$	0
				Total = 240	

Complexity: $\Theta(n \log n)$ — dominated by the sort; the scan is $\Theta(n)$.

Critical contrast, and a favourite exam point: with the **0/1** version of the same instance (no fractions allowed), the greedy answer would be items 1 and 2 = 160 with 20 kg wasted, whereas the true optimum is items 2 and 3 = **220**. **Greedy fails on 0/1 knapsack.** That problem needs dynamic programming (§7.1). The difference is that fractions restore the greedy-choice property.

7.1 0/1 Knapsack — work the table

Problem. n items; item i has weight w_i and value v_i . Capacity W . Each item is taken **entirely** or **not at all**. Maximise value.

Why greedy fails: shown in §6.1. There is no safe local choice — whether to take an item depends on what the rest of the capacity can hold.

Recurrence. Let $K[i][w]$ = maximum value obtainable using only the first i items with capacity w .

```
K[i][w] = 0                                     if i = 0 or w = 0
K[i][w] = K[i-1][w]                             if  $w_i > w$       (item
doesn't fit)
K[i][w] = max( K[i-1][w],   $v_i + K[i-1][w - w_i]$  )  otherwise
```

$\wedge \text{exclude } i$ $\wedge \text{include } i$

Worked example

$n = 4$, $W = 5$:

Item i	Weight w_i	Value v_i
1	2	3
2	3	4
3	4	5
4	5	6

The table $K[i][w]$, rows $i = 0..4$, columns $w = 0..5$:

$i \setminus w$	0	1	2	3	4	5
0 (no items)	0	0	0	0	0	0
1 ($w=2, v=3$)	0	0	3	3	3	3
2 ($w=3, v=4$)	0	0	3	4	4	7
3 ($w=4, v=5$)	0	0	3	4	5	7
4 ($w=5, v=6$)	0	0	3	4	5	7

Sample cells worked out longhand — show two or three of these in an exam answer:

- $K[1][2]$: $w_1 = 2 \leq 2$, so $\max(K[0][2], 3 + K[0][0]) = \max(0, 3+0) = 3$.
- $K[2][3]$: $w_2 = 3 \leq 3$, so $\max(K[1][3], 4 + K[1][0]) = \max(3, 4+0) = 4$.
- $K[2][5]$: $w_2 = 3 \leq 5$, so $\max(K[1][5], 4 + K[1][2]) = \max(3, 4+3) = 7$.
- $K[3][4]$: $w_3 = 4 \leq 4$, so $\max(K[2][4], 5 + K[2][0]) = \max(4, 5+0) = 5$.
- $K[3][5]$: $w_3 = 4 \leq 5$, so $\max(K[2][5], 5 + K[2][1]) = \max(7, 5+0) = 7$.
- $K[4][5]$: $w_4 = 5 \leq 5$, so $\max(K[3][5], 6 + K[3][0]) = \max(7, 6+0) = 7$.

Answer: maximum value = $K[4][5] = 7$.

Traceback — which items? Start at $K[4][5]$ and walk up:

At	Compare	Conclusion	Move to
$K[4][5] = 7$	$K[3][5] = 7$ — same	item 4 not taken	$K[3][5]$
$K[3][5] = 7$	$K[2][5] = 7$ — same	item 3 not taken	$K[2][5]$
$K[2][5] = 7$	$K[1][5] = 3$ — different	item 2 taken ; $w = 5 - 3 = 2$	$K[1][2]$
$K[1][2] = 3$	$K[0][2] = 0$ — different	item 1 taken ; $w = 2 - 2 = 0$	$K[0][0]$ — stop

Optimal set = {item 1, item 2}, total weight $2 + 3 = 5$, total value $3 + 4 = 7$. ✓

Complexity. Time $\Theta(nW)$, space $\Theta(nW)$ — reducible to $\Theta(W)$ with a single rolling row iterated *right to left* (so each item is used at most once).

The pseudo-polynomial point — a guaranteed exam mark. $\Theta(nW)$ *looks* polynomial, but W is a **number**, and its size in the input is $\log W$ bits. So the running time is exponential in the input *length*. That is why 0/1 knapsack is **NP-complete** and yet has an "efficient-looking" DP: the algorithm is **pseudo-polynomial**, not polynomial. See §11.

7.6 Dynamic programming vs divide and conquer — the exam table

	Divide and Conquer	Dynamic Programming
Subproblems	Independent / disjoint — they do not share sub-subproblems	Overlapping — the same subproblem recurs many times
Each subproblem solved	Once, by definition (they're distinct)	Once because it is stored ; recomputation is what DP eliminates
Direction	Top-down recursion	Typically bottom-up tabulation (or top-down with memoisation)
Storage	No table; only the recursion stack	A table of subproblem results
Requires	Optimal substructure	Optimal substructure + overlapping subproblems
Recombination	Combine subproblem answers (merge, etc.)	Choose the best among subproblem answers
Typical complexity	Often $\Theta(n \log n)$	Often $\Theta(n^2)$ or $\Theta(n^3)$ — but converts exponential to polynomial
Space	Usually small	Usually large (the table)
Examples	Binary search, merge sort, quick sort, Strassen, FFT, closest pair	0/1 knapsack, LCS, matrix chain, Floyd–Warshall, Bellman–Ford, optimal BST, edit distance, Fibonacci
If applied to the other's problem	D&C on Fibonacci → exponential blow-up from recomputation	DP on merge sort → wasted table, no benefit, since no subproblem repeats

The one-line answer: *both* split a problem into subproblems, but divide and conquer's subproblems are disjoint so recursion suffices, whereas dynamic programming's subproblems overlap, so it stores results to avoid exponential recomputation.

7.7 Greedy vs dynamic programming

	Greedy	Dynamic Programming
Decision making	Makes one choice at each step and never revisits it	Considers all choices at each step and picks the best
Direction	Top-down: choose first, then solve the subproblem	Bottom-up: solve subproblems first, then choose
Requires	Optimal substructure + greedy-choice property	Optimal substructure + overlapping subproblems
Optimality	Only for problems with the greedy-choice property	Always optimal when the recurrence is correct
Speed	Faster — usually $\Theta(n \log n)$	Slower — usually $\Theta(n^2)/\Theta(n^3)$
Space	$\Theta(1)$ extra typically	Table required
The classic contrast	Fractional knapsack: $\Theta(n \log n)$	0/1 knapsack: $\Theta(nW)$

8.1 Backtracking

Concept. Backtracking is organised, intelligent exhaustive search. You build a solution one component at a time, and **as soon as a partial solution cannot possibly be extended to a valid complete solution, you abandon it and back up** — "prune the branch". The saving comes from cutting whole subtrees of the search space without exploring them.

The search space is conceptually a **state-space tree**: the root is the empty solution; each level fixes one more component; the leaves are complete candidates. Backtracking is a **DFS of that tree with pruning by a bounding/feasibility function**.

Vocabulary to use in an answer:

Term	Meaning
State-space tree	Tree of all partial solutions
Live node	A generated node whose children have not all been explored
E-node (expanding node)	The live node currently being expanded
Dead node	A node that is not expanded further — either infeasible, or fully explored
Bounding function	The test that decides a node is dead
Promising / non-promising node	Can / cannot lead to a solution

Two formulations: *implicit constraints* (rules the components must satisfy relative to each other) and *explicit constraints* (the domain each component may take values from).

n-Queens

Problem. Place n queens on an $n \times n$ board so that no two attack each other (no shared row, column or diagonal).

Formulation. One queen per row is forced, so the solution is a vector $x[1..n]$ where $x[i]$ is the **column** of the queen in row i . That single observation collapses the search space from $C(n^2, n)$ to n^n , and pruning cuts it far further.

Feasibility test — queens in rows i and j clash if:

- $x[i] == x[j]$ — same column, **or**
- $|x[i] - x[j]| == |i - j|$ — same diagonal.

```

N-QUEENS(k, n)           // place queens in rows k..n
  for col = 1 to n:
    if PLACE(k, col):
      x[k] = col
      if k == n: print x           // a complete solution
    else: N-QUEENS(k+1, n)
  // no "undo" needed: x[k] is overwritten on the next iteration

```

Worked trace — 4-Queens. Columns 1–4, rows 1–4.

Step	Partial x	Action
1	$x_1 = 1$	Row 1: queen at column 1
2	$x_2 = 1?$	Same column — reject. $x_2 = 2?$ Diagonal (
3	$x_3 = 1?$ col clash. $2?$ diag with x_2 (	2–3
4	$x_2 = 4$	Try next column in row 2: $x_2 = 4$ ✓
5	$x_3 = 2$	1? col clash. 2 ✓ (no clash with (1,1) or (2,4))

6	$x_4 = ?$	1,2 clash by column; 3 diag with x_3 (
7	$x_3 = 3?$	col clash with $x_2=4$? no... but 3 vs $x_2=4$:
8	$x_1 = 2$	Restart row 1 at column 2
9	$x_2 = 4$	1? diag(
10	$x_3 = 1$	1 ✓
11	$x_4 = 3$	3 ✓ — SOLUTION: (2, 4, 1, 3)

The two solutions for $n = 4$ are **(2,4,1,3)** and **(3,1,4,2)** — mirror images. Board for (2,4,1,3):

```

      c1  c2  c3  c4
r1    .   Q   .   .
r2    .   .   .   Q
r3    Q   .   .   .
r4    .   .   Q   .

```

Number of solutions: $n=4 \rightarrow 2$, $n=5 \rightarrow 10$, $n=6 \rightarrow 4$, $n=8 \rightarrow 92$. Worst-case complexity $O(n!)$ — backtracking prunes heavily but does not change the exponential class.

Graph colouring (m-colouring)

Problem. Can the vertices of a graph be coloured with at most m colours so that no two adjacent vertices share a colour? (The **chromatic number** $\chi(G)$ is the smallest such m .)

```

M-COLOUR(k)                                // colour vertex k
  for c = 1 to m:
    if all neighbours of k already coloured have colour  $\neq$  c:
      x[k] = c
      if k == n: print x
    else:      M-COLOUR(k+1)

```

State-space tree has m^n leaves; complexity **$O(m^n \cdot n)$** .

Facts to know: a graph is 2-colourable iff it is **bipartite** iff it contains no odd cycle (testable in $O(V+E)$ by BFS — this special case is *not* NP-hard). A complete graph K_n needs n colours. Deciding 3-colourability is **NP-complete**. **Four Colour Theorem:** every planar graph is 4-colourable. Applications: register allocation in compilers ([CSD_02 §15](#)), exam timetabling, frequency assignment, Sudoku.

Hamiltonian cycle

Problem. A cycle visiting **every vertex exactly once** and returning to the start.

Distinguish clearly from Eulerian:

	Eulerian circuit	Hamiltonian cycle
Visits every...	edge exactly once	vertex exactly once
Existence test	Easy: connected and every vertex has even degree	NP-complete — no easy criterion
Complexity	$O(V+E)$ (Fleury/Hierholzer)	Exponential

Backtracking approach: build the path vertex by vertex; $x[k]$ is valid if it is adjacent to $x[k-1]$ and not already in the path; at $k = n$ also require $x[n]$ adjacent to $x[1]$. Complexity $O(n!)$.

Sufficient conditions worth naming: *Dirac* — if every vertex has degree $\geq n/2$ ($n \geq 3$), the graph is Hamiltonian. *Ore* — if $\deg(u) + \deg(v) \geq n$ for every non-adjacent pair, likewise.

Other classic backtracking problems

Problem	Description
Sum of subsets	Find subsets of a set summing to a target m
Sudoku	Constraint satisfaction on a 9×9 grid
Rat in a maze / knight's tour	Path finding on a grid
Permutation generation	All orderings of n items
Crossword / word puzzles	Constraint satisfaction

11.5 The classic NP-complete problems — know these

Problem	Statement (decision version)
SAT	Is a given Boolean formula satisfiable? (<i>the root</i>)
3-SAT	Same, restricted to CNF with exactly 3 literals per clause. (<i>2-SAT, by contrast, is in P — a favourite trap</i>)
CLIQUE	Does G contain a complete subgraph on k vertices?
VERTEX COVER	Is there a set of k vertices touching every edge?
INDEPENDENT SET	Is there a set of k mutually non-adjacent vertices? (<i>complement of vertex cover</i>)
SUBSET SUM	Does some subset of a set of integers sum to exactly t ?
PARTITION	Can a set of integers be split into two subsets of equal sum?
0/1 KNAPSACK (decision)	Is there a subset of weight $\leq W$ and value $\geq v$?
HAMILTONIAN CYCLE / PATH	Does G contain a cycle/path visiting every vertex exactly once?
TSP (decision)	Is there a tour of total cost $\leq k$?
GRAPH COLOURING ($k \geq 3$)	Is G k -colourable? (<i>2-colourability is in P</i>)
SET COVER	Can k of the given sets cover the universe?
BIN PACKING	Can the items fit into k bins of given capacity?
JOB SCHEDULING with deadlines and penalties	Various forms
SUDOKU (generalised $n^2 \times n^2$)	Is a partially filled grid completable?

Deceptive near-neighbours — the pairs examiners love:

In P (easy)	NP-complete (hard)
2-SAT	3-SAT
2-colourability (bipartiteness)	3-colourability
Eulerian circuit	Hamiltonian cycle

Shortest path	Longest simple path
Minimum spanning tree	Travelling salesman / Steiner tree
Fractional knapsack	0/1 knapsack
Matching in a bipartite graph	3-dimensional matching
Primality testing	Integer factorisation (believed hard; not known NP-complete)
Linear programming	Integer linear programming

11.6 The P vs NP question

Is $P = NP$? Is every problem whose solution can be *checked* quickly also a problem whose solution can be *found* quickly?

- One of the seven **Clay Millennium Prize Problems** — **US\$1,000,000** for a proof either way.
- Open since 1971. The overwhelming consensus is $P \neq NP$, but there is no proof.
- **To prove $P = NP$:** find a polynomial-time algorithm for any single NP-complete problem.
- **To prove $P \neq NP$:** prove that no polynomial-time algorithm exists for some problem in NP — a lower-bound proof, which is far harder to establish than an algorithm.

Consequences if $P = NP$ (this was 2025 MCQ Q34):

- Every NP-complete problem — thousands of optimisation, scheduling, routing, design and logistics problems — becomes efficiently solvable. Enormous practical gains.
- **Most public-key cryptography collapses.** RSA rests on factoring being hard; much of the rest rests on problems in NP. (Strictly: factoring is not known to be NP-complete, but $P = NP$ would place it in P as well, since it is in NP.)
- Mathematics itself changes: finding a proof of bounded length becomes as easy as checking one, so automated theorem proving becomes routine.

Consequences if $P \neq NP$: the status quo is confirmed — we must live with approximation algorithms, heuristics and exponential exact algorithms for hard problems, and cryptography's foundations are (conditionally) safe.

11.7 What to do when your problem is NP-complete

An exam answer that stops at "it's hard" is incomplete. Say what an engineer actually does:

Strategy	Description	Example
Approximation algorithms	Guarantee a solution within a proven factor of optimal in polynomial time	Vertex cover: 2-approximation by taking both endpoints of a maximal matching. Metric TSP: 1.5-approximation (Christofides)
Heuristics	No guarantee, but good in practice	Greedy, local search, simulated annealing, genetic algorithms, ant colony optimisation
Pseudo-polynomial algorithms	Polynomial in the numeric <i>value</i> of an input, not its bit length	0/1 knapsack DP in $\Theta(nW)$ — see §7.1
Parameterised / fixed-parameter tractable	Exponential only in a small parameter k , e.g. $O(2^k \cdot n)$	Vertex cover with small k

Restrict the input	The problem may be in P on a special class	3-colouring is hard in general, easy on interval graphs; many problems are polynomial on trees or planar graphs
Exact exponential with good pruning	Branch and bound, ILP solvers, SAT solvers	Modern SAT solvers routinely handle millions of variables despite the theory
Randomised algorithms	Accept a small probability of error for speed	Miller–Rabin primality

🕒 Forensic — Linux & macOS artifacts

Source: ComputerForensic/FOR_02_SystemArtifacts.md

4.4 ext2 / ext3 / ext4 (Linux)

Core concept: **the inode**. See §6.2 for the full treatment. Briefly:

Feature	ext2	ext3	ext4
Journaling	No	Yes	Yes
Extents (vs block pointers)	No	No	Yes — better for large files, less fragmentation
Max file size	2 TB	2 TB	16 TB
Max volume size	32 TB	32 TB	1 EB
Delayed allocation	No	No	Yes
Nanosecond timestamps	No	No	Yes (plus a creation time field, <code>crtime</code> , not exposed by <code>stat</code> in all versions)
Deleted-file recovery	Comparatively easy — block pointers often survive	Harder	Hardest — ext3/ext4 zero the block pointers / extent tree in the inode on delete

Key exam fact: ext3 and ext4 wipe the inode's data-block references when a file is deleted, so classical "undelete via inode" fails and you must rely on **carving** or on the journal. ext2 does not, which is why old Linux textbooks make undeletion sound easy.

Other Linux/Unix file systems worth naming: **XFS**, **Btrfs** (copy-on-write, snapshots — good for forensics because old versions persist), **ZFS**, **ReiserFS** (historical), **F2FS** (flash-friendly, used on some Android devices), and **JFS**.

4.5 HFS+ and APFS (macOS)

Aspect	HFS+ ("Mac OS Extended", 1998)	APFS (Apple File System, 2017)
Core structure	Catalog File — a B-tree of all files and folders, keyed by Catalog Node ID (CNID)	Container holding multiple volumes that share a free-space pool
Journaling	Yes (HFS+J)	Metadata via copy-on-write, no traditional journal

Copy-on-write	No	Yes — never overwrites live data in place
Snapshots	No (Time Machine used hard links)	Yes, native — highly valuable evidence source
Encryption	FileVault (whole disk, CoreStorage)	Native per-file and per-volume encryption (FileVault 2)
Timestamps	1-second resolution	Nanosecond
Space sharing	Fixed partitions	Volumes grow/shrink dynamically inside a container
Cloning	No	Instant file clones (shared blocks)
Forensic note	Deleted-file recovery reasonably tractable	Copy-on-write means old versions may persist , but encryption-by-default and the container layout complicate acquisition; tool support is still less mature

Other Apple structures to name: **Extents Overflow File**, **Attributes File** and **Allocation**

File in HFS+; **resource forks** (the historical Mac analogue of NTFS ADS); the **.DS_Store** file in every browsed folder (records folder view settings — proves a folder was opened).

§6.9 Likely exam questions (Linux)

Marks	Question
5	What is an inode? What information does it contain?
5	Explain the seven fields of <code>/etc/passwd</code> .
5	Distinguish between <code>atime</code> , <code>mtime</code> and <code>ctime</code> .
5	What is the SUID bit? Why is it a security and forensic concern?
5	How are hidden files implemented in Linux?
5	Distinguish between <code>wtmp</code> , <code>btmp</code> and <code>lastlog</code> .
10	Describe the Linux file-system hierarchy and identify the directories of forensic interest.
10	Discuss the log files found in <code>/var/log</code> and their evidentiary value in investigating an intrusion.
10	Explain Linux file ownership and permissions, including the special bits, with examples.
15	You are given a disk image of a compromised Linux server. Describe systematically the artifacts you would examine and what each would tell you.

MCQ traps (Linux)

- **wtmp** = successful logins (read with `last`); **btmp** = failed (read with `lastb`); **lastlog** = last login per user; **utmp** = currently logged in. The most-swapped set in the paper.
- `/etc/shadow` holds the hashes; `/etc/passwd` holds the `x` placeholder.
- **\$6\$** = SHA-512, **\$1\$** = MD5, **\$2y\$** = bcrypt, **\$5\$** = SHA-256.
- **ctime** is **change time (inode metadata)**, **NOT creation time**. The commonest trap in the entire Linux section.
- **UID 0** = **root**, regardless of the account name.
- **/proc** is **virtual and exists only in memory** — it is not in a dead-box image.

- **ext2 undelete is easier than ext3/ext4** because ext3/4 zero the inode block pointers.
- Hidden files in Linux = a leading dot, **a convention in `ls`, not a file-system attribute**.
- The **sticky bit on a directory** restricts deletion to the file's owner (`/tmp`).

§7. macOS (Mac OS X) Systems and Artifacts

Less likely than Windows in the MCQ, but the syllabus names it explicitly, so a 5-mark question is entirely plausible. Learn the locations table and the four "signature" artifacts: **plists, Keychain, Spotlight and FSEvents**.

§7.6 Likely exam questions (macOS)

Marks	Question
5	What is a plist file? Where are plists found on macOS?
5	What is <code>launchd</code> ? Name the directories used for automatic startup.
5	Write a note on the macOS Keychain from a forensic point of view.
5	What are FSEvents and why are they forensically important?
10	Describe the principal user artifacts on a macOS system and their locations.
10	Compare Windows, Linux and macOS with respect to hidden files, startup persistence and system logging.
15	Discuss macOS system and user artifacts under the headings: startup and services, network configuration, hidden directories, system logs and user artifacts.

MCQ traps (macOS)

- **`launchd` is PID 1** on macOS (not `init`, not `systemd`).
- **Binary plists start with `bplist00`** and must be converted before reading.
- **`.DS_Store` records folder view settings** and proves a folder was opened in Finder.
- **Spotlight index = `.Spotlight-V100`; FSEvents = `.fsevents`**. Do not swap.
- **`com.apple.quarantine` is the macOS equivalent of the NTFS `Zone.Identifier` stream**.
- **APFS is copy-on-write with native snapshots**; HFS+ is not.
- The default shell on modern macOS is **`zsh`**, not `bash`.

§8. Cross-platform artifact comparison — one table for revision

Question	Windows	Linux	macOS
Config store	Registry (<code>SYSTEM</code> , <code>SOFTWARE</code> , <code>SAM</code> , <code>NTUSER.DAT</code>)	Text files in <code>/etc</code> and dot-files in <code>~</code>	plists in <code>/Library</code> and <code>~/Library</code>
Init / PID 1	<code>wininit</code> / <code>services.exe</code>	<code>init</code> / <code>systemd</code>	<code>launchd</code>

Persistence	Run keys, Services, Scheduled Tasks, Startup folder, WMI	<code>cron</code> , systemd timers/ services , <code>rc.local</code> , <code>.bashrc</code> , <code>/etc/init.d</code>	LaunchDaemons / LaunchAgents , Login Items
Authentication log	<code>Security.evtx</code> (4624/4625)	<code>auth.log</code> / secure , <code>wtm</code> , <code>btm</code>	Unified log, <code>/var/log/system.log</code> , ASL
Hidden files	H/S attributes + ADS	Leading dot (a convention)	Leading dot + hidden flag + <code>~/Library</code> hidden by Finder
"Downloaded from the internet" marker	<code>:Zone.Identifier</code> ADS	(none standard)	<code>com.apple.quarantine</code> xattr + QuarantineEvents DB
Recycle bin	<code>C:\\$Recycle.Bin\<SID>\\$I\$R</code>	<code>~/ .local/share/Trash/{files,info}</code>	<code>~/ .Trash</code> , <code>~/ .Trashes/<uid></code>
Recent files	RecentDocs, LNK, Jump Lists	<code>.recently-used.xbel</code> , <code>.viminfo</code>	<code>*.sfl2</code> shared file lists
Folder-browsing proof	ShellBags	(weak — mount logs, shell history)	<code>.DS_Store</code>
Program execution proof	Prefetch , UserAssist, ShimCache, Amcache, BAM	<code>auth.log/audit.log</code> , <code>.bash_history</code> , package logs	Unified log, <code>ExecPolicy</code> , crash reports, quarantine
Thumbnail cache	<code>thumbcache_*.db</code>	<code>~/ .cache/thumbnails/</code>	Quick Look thumbnails, Photos DB
Memory on disk	<code>pagefile.sys</code> , <code>swapfile.sys</code> , hiberfil.sys	swap partition / swapfile	<code>/private/var/vm/swapfile*</code> , <code>sleepimage</code>
Snapshots / previous versions	Volume Shadow Copies	LVM/Btrfs/ZFS snapshots	APFS snapshots , Time Machine, <code>.DocumentRevisions-V100</code>
Stored credentials	SAM hashes, LSA secrets, DPAPI , Credential Manager	<code>/etc/shadow</code> , <code>~/ .ssh/</code> , keyrings (GNOME Keyring/KWallet)	Keychain
File-change journal	<code>\$UsnJrnl</code> / <code>\$LogFile</code>	<code>auditd</code> , ext4 journal	FSEvents

§9. Pointers to the CORE files — TP-I Units 6, 8 and 9

These three units are **shared-core engineering** covered in `..\_Shared\`. **Do not study a second version of them here.** What follows is only the *forensic slant* that the shared notes will not give you, plus the exam angle specific to this elective.

⚠ Verify the exact CORE filenames against `C:\Users\hp\Desktop\prep\Materials\_Shared\`.